Sesión 5

Fragmentación por Regiones y Carga Masiva

Se sigue trabajando sobre la base juego_online (Módulos 1–4). Se resuelven tres problemas con tres scripts SQL, más un apartado conceptual sobre la nube. El orden es obligatorio: primero se organizan los datos (particionar), luego se optimiza el acceso (indexar) y al final se alimenta la tabla con volumen (ETL).

- **Script 1** → 01_fragmentacion_particiones_region.sql
- **Script 2** → 02_indices_matchmaking.sql
- **Script 3** → 03_etl_carga_masiva.sql

Script 1 — Fragmentación por región

separar físicamente la tabla de partidas según la región del jugador (América / Europa), como si cada región residiera en su propio servidor

código

sql

```
CREATE TABLE Partidas_Fragmentada (
  ...
  region VARCHAR(20) NOT NULL,
  ...
  PRIMARY KEY (partida_id, region)
) ENGINE=InnoDB
PARTITION BY LIST COLUMNS(region) (
  PARTITION p_america VALUES IN ('AMERICA'),
  PARTITION p_europa  VALUES IN ('EUROPA')
);
```

- PARTITION BY LIST COLUMNS(region) → el motor guarda las filas de 'AMERICA' en la partición p_america y las de 'EUROPA' en p_europa, aunque para el usuario la tabla siga viéndose como una sola.
- PRIMARY KEY (partida_id, region) → regla obligatoria del motor: toda clave única debe incluir la columna de partición.
- **Sin** FOREIGN KEY → MySQL/MariaDB no admiten claves foráneas en tablas particionadas. La integridad referencial pasa a la aplicación. Esta es la lección central del módulo.

Migración de datos

sql

```
INSERT INTO Partidas_Fragmentada (...)
SELECT p.jugador_id, p.juego_id, j.region, ...
FROM Partidas p
JOIN Jugadores j ON j.jugador_id = p.jugador_id;
```

La región no existe en Partidas; se toma del jugador (j.region). Esto denormaliza el dato a cambio de poder particionar.

Comprobación

sql

```
EXPLAIN SELECT * FROM Partidas_Fragmentada WHERE region = 'AMERICA';
```

La columna partitions del EXPLAIN debe mostrar únicamente p_america → **partition pruning**: el motor nunca abre la partición que no necesita.

Script 2 — Índices para matchmaking

la consulta de matchmaking (buscar jugadores de nivel similar en la misma región) responda en milisegundos en vez de escanear miles de filas.

Los tres índices

sql

```
CREATE INDEX idx_partidas_frag_region_fecha
ON Partidas_Fragmentada (region, fecha DESC);   -- filtro por región + orden por fecha
```

- Índice compuesto con el orden correcto: primero la igualdad (region), luego el rango/orden (fecha DESC).
- Gracias a esto, el EXPLAIN cambia de type: ALL a type: ref o range.

Script 3 — ETL de carga masiva

Propósito: simular la importación de un CSV con miles de partidas históricas, usando una CTE recursiva (equivalente educativo a LOAD DATA INFILE).

Generación de 5000 filas

sql

```
INSERT INTO Partidas_Fragmentada (...)
```

```sql
WITH RECURSIVE secuencia AS (
  SELECT 1 AS n
  UNION ALL
  SELECT n + 1 FROM secuencia WHERE n < 5000
)
SELECT
  j.jugador_id,
  FLOOR(RAND() * 2) + 1                AS juego_id,      -- juego 1 o 2
  j.region,                              -- región del jugador
  FLOOR(RAND() * 9000)                 AS puntaje,       -- 0 a 8999
  FLOOR(RAND() * 900) + 60               AS duracion_seg,  -- 60 a 959 s
  ELT(FLOOR(RAND() * 2) + 1, 'victoria', 'derrota'),
  NOW() - INTERVAL FLOOR(RAND() * 180) DAY
FROM secuencia
JOIN Jugadores j
  ON j.jugador_id = ((secuencia.n MOD (SELECT COUNT(*) FROM Jugadores)) + 1);
```

- **CTE recursiva** secuencia → genera los números del 1 al 5000 (caso base 1, caso recursivo n + 1).
- RAND() → solo afecta a puntaje, duracion_seg, resultado y fecha (no a jugador ni región).
- MOD COUNT(*) → reparte las 5000 filas en ciclo entre los 5 jugadores (1000 cada uno). Como 3 son de América y 2 de Europa → **3000 / 2000 exactos**, siempre.

Comprobación

sql

```sql
SELECT region, COUNT(*) AS total_partidas
FROM Partidas_Fragmentada
GROUP BY region;
```

Resultado esperado: AMERICA = 3000, EUROPA = 2000.

Nota MySQL 8: si aparece error de recursión, descomentar SET SESSION cte_max_recursion_depth = 10000; (MySQL limita a 1000 niveles por defecto; MariaDB no).

Paso 4 — Comandos de terminal (no SQL)

Referencia conceptual. **No** se pegan en phpMyAdmin.

```
bash
mysqldump -u root -p juego_online Partidas_Fragmentada > historico_partidas.sql

aws s3 cp historico_partidas.sql s3://mi-bucket-juego/backups/
aws s3 cp s3://mi-bucket-juego/import/nuevas_partidas.csv ./nuevas_partidas.csv

mysql -u root -p juego_online -e "LOAD DATA INFILE './nuevas_partidas.csv' INTO TABL
E Partidas_Fragmentada ..."
```

- **Carga (upload):** exportar con `mysqldump` y subir a S3 / Azure.
- **Descarga (download):** bajar un CSV de la nube y cargarlo con `LOAD DATA INFILE` o la pestaña *Importar* de phpMyAdmin.
- Es el mismo patrón **Extract → Transform → Load** del Script 3, solo que el origen/destino es almacenamiento remoto.

Orden de ejecución

```
text
Script 1 → crea Partidas_Fragmentada y migra
Script 2 → indexa la tabla creada
Script 3 → inserta 5000 filas en la tabla indexada
```

Si se altera el orden:

- los scripts fallan (tabla inexistente), o
- se pierden datos (el `DROP TABLE` del Script 1 borra las 5000 filas si se vuelve a ejecutar).

Resumen

- `PARTITION BY LIST COLUMNS(region)` divide físicamente por región; el usuario ve una sola tabla.
- La PK debe incluir la columna de partición (`partida_id, region`) — regla del motor.
- Sin FKs en tablas particionadas → la integridad se controla en la aplicación.
- Índices compuestos con orden **igualdad → rango/ordenación** transforman `type: ALL` en `type: ref/range`.

- La CTE recursiva genera 5000 filas sin archivos externos; MOD garantiza el reparto 3000/2000.
- El patrón **ETL** es el mismo en local (phpMyAdmin) y en la nube (S3, Azure, Glue).